HDOC Day-54

Maximum Sum of a Contiguous Subarray of Size k

Problem Statement

Read the size and elements of a variable-length integer array and an integer k satisfying 0 < k < size. Pass the array by address to max_subarray_sum(). Find the maximum sum among all contiguous subarrays of length k; display -1 for an invalid k.

Algorithm

1. Start. Read a positive integer size and declare int arr[size].

2. Read size integer elements into arr.

3. Read k and pass arr, size and k to max_subarray_sum(arr, size, k).

4. If k <= 0 or k >= size, print -1 and return.

5. Calculate the sum of the first k elements; store it as maximum.

6. Slide the window one position at a time: subtract the element leaving the window and add the new element.

7. Update maximum whenever the current window sum is larger.

8. Display the maximum subarray sum and stop.

Test Case Table

TC	Size	Elements	k	Expected	Purpose
1	6	2 1 5 1 3 2	3	9	Professor example 1
2	5	2 3 4 1 5	2	7	Professor example 2
3	3	1 2 3	4	-1	k greater than size
4	4	4 1 2 3	0	-1	k is zero
5	4	4 1 2 3	-1	-1	Negative k
6	4	4 1 2 3	4	-1	k equals size (invalid per task)
7	5	-5 -2 -8 -1 -3	2	-4	All negative numbers
8	5	1 1 1 1 1	3	3	Repeated values
9	2	10 -2	1	10	Smallest valid k
10	1	7	1	-1	Single-element array; no valid k

C Program

```
#include <stdio.h>

void max_subarray_sum(const int *arr, int size, int k)
{
    int i;
    long long sum = 0, maximum;

    if (k <= 0 || k >= size)
    {
```

```
        printf("Maximum subarray sum: -1\n");
        return;
    }

    for (i = 0; i < k; i++)
        sum += *(arr + i);

    maximum = sum;

    for (i = k; i < size; i++)
    {
        sum = sum - *(arr + i - k) + *(arr + i);
        if (sum > maximum)
            maximum = sum;
    }

    printf("Maximum subarray sum: %lld\n", maximum);
}

int main(void)
{
    int size, k, i;

    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000)
    {
        printf("Invalid size. Enter a positive size up to 10000.\n");
        return 1;
    }

    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid array element.\n");
            return 1;
        }
    }

    printf("Enter k: ");
    if (scanf("%d", &k) != 1)
    {
        printf("Invalid k.\n");
        return 1;
    }

    max_subarray_sum(arr, size, k);
    return 0;
}
```

Compilation and Execution

```
gcc -std=c99 hdoc54.c -o hdoc54

./hdoc54
```

Compilation and Execution

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc54.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc54.c -o hdoc54
```

Test Case 1

Input: size = 6; elements = 2 1 5 1 3 2; k = 3 | Expected result: 9

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 6
Enter 6 array elements:
2 1 5 1 3 2
Enter k: 3
Maximum subarray sum: 9
```

Test Case 2

Input: size = 5; elements = 2 3 4 1 5; k = 2 | Expected result: 7

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 5
Enter 5 array elements:
2 3 4 1 5
Enter k: 2
Maximum subarray sum: 7
```

Test Case 3

Input: size = 3; elements = 1 2 3; k = 4 | Expected result: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 3
Enter 3 array elements:
1 2 3
Enter k: 4
Maximum subarray sum: -1
```

Test Case 4

Input: size = 4; elements = 4 1 2 3; k = 0 | Expected result: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 4
Enter 4 array elements:
4 1 2 3
Enter k: 0
Maximum subarray sum: -1
```

Test Case 5

Input: size = 4; elements = 4 1 2 3; k = -1 | Expected result: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 4
Enter 4 array elements:
4 1 2 3
Enter k: -1
Maximum subarray sum: -1
```

Test Case 6

Input: size = 4; elements = 4 1 2 3; k = 4 | Expected result: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 4
Enter 4 array elements:
4 1 2 3
Enter k: 4
Maximum subarray sum: -1
```

Test Case 7

Input: size = 5; elements = -5 -2 -8 -1 -3; k = 2 | Expected result: -4

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 5
Enter 5 array elements:
-5 -2 -8 -1 -3
Enter k: 2
Maximum subarray sum: -4
```

Test Case 8

Input: size = 5; elements = 1 1 1 1 1; k = 3 | Expected result: 3

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 5
Enter 5 array elements:
1 1 1 1 1
Enter k: 3
Maximum subarray sum: 3
```

Test Case 9

Input: size = 2; elements = 10 -2; k = 1 | Expected result: 10

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 2
Enter 2 array elements:
10 -2
Enter k: 1
Maximum subarray sum: 10
```

Test Case 10

Input: size = 1; elements = 7; k = 1 | Expected result: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc54
Enter array size: 1
Enter 1 array elements:
7
Enter k: 1
Maximum subarray sum: -1
```